Learning Objectives

- Understand the memory layout of struct and union types
- Investigate alignment and padding
- Understand byte order and Little Endian
- Use unions to represent alternative data layouts
- Extend our expression-tree representation with function calls and unary nodes

Keeping Your Work

Keep all the example programs and other files you create in this worksheet in a directory such as `~/hpc0/session16`.

This is primarily a Top-Down worksheet, so most experiments are separate from our compiler project. Whenever a task requires you to modify the compiler project itself, we will explicitly tell you to work in your not-abc repository instead.

Step 1 — First Contact with `union`

In this session, we will investigate how `struct` and `union` types are laid out in memory. As a first example, we use a small data type that allows us to look at the bit representation of a double. Create a file `xtest_union.abc` in your `~/hpc0/session16` directory:

```
@ <stdio.hdr>

struct IEEE_754
{
    union {
        d: double;
        i: u64;
    };
};

global x: IEEE_754 = { .d = 1};

fn main()
{
    printf("x.d = %g\n", x.d);
    printf("x.i = 0x%016llX = %llu\n", x.i, x.i);

    printf("Some Info:\n");
    printf("sizeof(x) = %zu\n", sizeof(x));
    printf("sizeof(IEEE_754) = %zu\n", sizeof(IEEE_754));
    printf("&x   = %p\n", &x);
    printf("&x.d = %p\n", &x.d);
    printf("&x.i = %p\n", &x.i);
}
```

If some of the `printf` format specifiers are unfamiliar, here is a quick overview:

- `%g` prints a floating-point value (`double`) in a compact form.
- `%llu` prints a u64 value as an unsigned decimal integer (`unsigned long long`).
- `%llX` prints the same value in hexadecimal using uppercase digits (`A`–`F`). In `%016llX`, the `16` specifies a width of 16 characters and the `0` adds leading zeros if necessary.
- `%zu` prints a value of type `size_t`, such as the result of `sizeof`.
- `%p` prints a pointer value, which we use here to inspect addresses.

The initializer

```
global x: IEEE_754 = { .d = 1};
```

specifies which member should be initialized. Here, `x.d` is initialized with the value `1`.

Compile and run the program. Your exact addresses will probably be different, but the important parts of the output should look similar to

Although `IEEE_754` contains two members, an object of this type occupies only **8 bytes**. Even more interestingly, compare the three addresses `&x`, `&x.d`, and `&x.i`.

© Michael C. Lehn | CC BY-SA 4.0

Compile and run the program. Your exact addresses will probably be different, but the important parts of the output should look similar to

```
x.d = 1
x.i = 0x3FF0000000000000 = 4607182418800017408
Some Info:
sizeof(x) = 8
sizeof(IEEE_754) = 8
&x   = ...
&x.d = ...
&x.i = ...
```

Although `IEEE_754` contains two members, an object of this type occupies only 8 bytes. Even more interestingly, compare the three addresses `&x`, `&x.d`, and `&x.i`.

Explain what you observe: Draw a small sketch of how `x`, `x.d`, and `x.i` are laid out in memory.

Now comment out the union block — but leave its two members in place:

```
struct IEEE_754
{
    // union {
        d: double;
        i: u64;
    //}
};
```

Compile and run the program again. Compare `sizeof(IEEE_754)`, `&x.d`, and `&x.i` with the previous results.

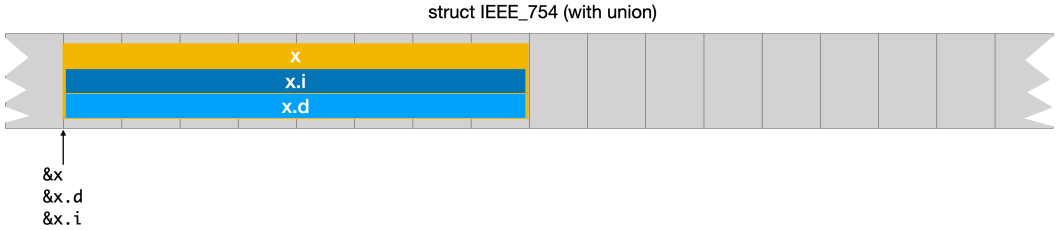
How has the memory layout changed: Draw another sketch and compare it with your first one.

With the union, `d` and `i` start at the same address and therefore occupy the same region of memory. The same 8 bytes can be interpreted either as a double through `x.d` or as a `u64` through `x.i`.

Without the `union`, `d` and `i` are ordinary structure members and occupy separate regions of memory.

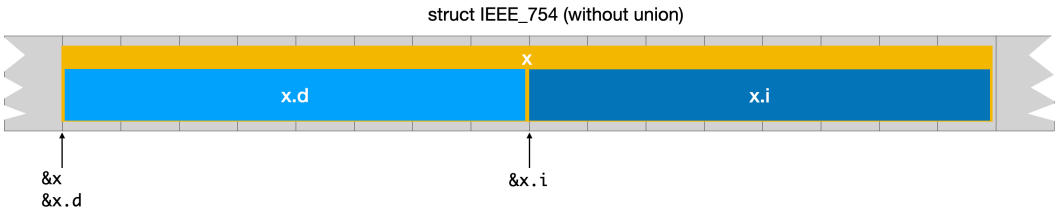
Spoiler: Compare your sketch with the one below.

We will use this way of drawing memory layouts throughout this worksheet. Make sure you understand how the addresses and the overlapping memory regions are represented.



With the `union`, `d` and `i` start at the same address.

Again, compare your sketch with the one below.



Without the `union`, `d` and `i` occupy consecutive memory regions instead of sharing the same 8 bytes.

© Michael C. Lehn | CC BY-SA 4.0

A Note on Going Deeper

We will not study `IEEE 754` in detail here; we only use it as a convenient example for interpreting the same 8 bytes in different ways. Likewise, the next step will only scratch the surface of `Little Endian` vs. `Big Endian`.

This is intentional: one goal of HPC0 is to give you enough foundations — and hopefully enough curiosity — to explore such topics on your own. *The general motto is: first understand the problem, then be able to find and understand the solutions when you need them*.

Step 2 — In Which Order Are Bytes Stored in Memory?

Extend `IEEE_754` by another `union` member:

```
struct IEEE_754
{
    union {
        d: double;
        i: u64;
        c: array[8] of char;
    }
};
```

Then add the following loop to `main`:

```
for (local i: int = 0; i < 8; ++i) {
    printf("x.c[%d] = %02X\n", i, x.c[i]);
}
```

The array `x.c` gives us a convenient way to inspect the eight individual bytes one after another.

Tasks

1. Draw the eight bytes occupied by `x` as eight consecutive boxes in memory, with addresses increasing from left to right.
2. Enter the hexadecimal values printed for `x.c[0]` through `x.c[7]` into your drawing.
3. Compare their order with the hexadecimal representation of `x.i`. Which byte of `x.i` is stored at the lowest memory address?

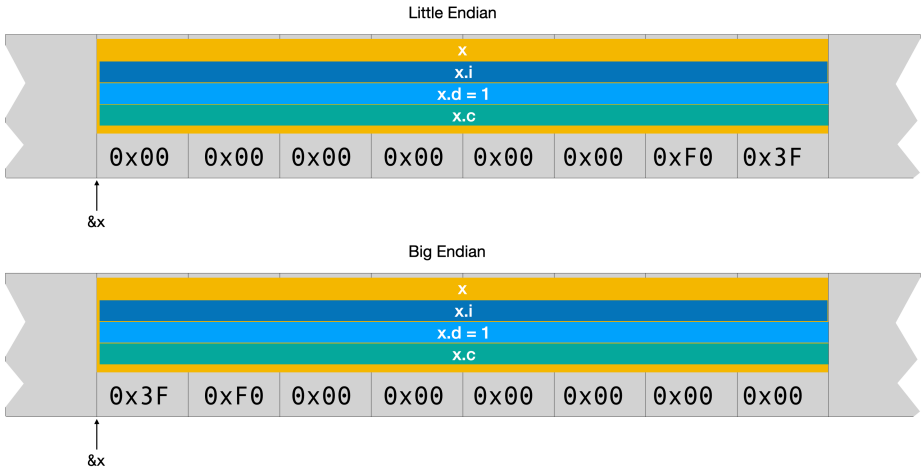
Little Endian and Big Endian

There are two common conventions for storing multi-byte values:

- **Little Endian:** the least significant byte is stored at the lowest address.
- **Big Endian:** the most significant byte is stored at the lowest address.

Which one does your machine use?

Compare your memory-layout sketch with the one below. Make sure you can relate the order of the individual bytes in memory to the hexadecimal representation of `x.i`.



© Michael C. Lehn | CC BY-SA 4.0

Step 3 — Back to Our Compiler: The Memory Layout of `Expr`

Now let us return to our compiler project. Change into your `not-abc` repository. In `expr.hdr`, our expression-tree nodes are currently represented by

```
struct Expr
{
    kind: ExprKind;
    binary:     struct Binary {
        left, right: ->Expr;
    };
    primary: struct Primary {
        decimal: ->UStr;
    };
};
```

Before changing this representation, let us first investigate how much memory it actually occupies and how its members are laid out.
Create a new test program `xtest_expr_layout.abc` in your `not-abc` repository:

```
@ <stdio.hdr>
@ "expr.hdr"

fn main()
{
    local e: Expr;
    printf("sizeof(Expr) = %zu\n", sizeof(e));
    printf("offset kind = %td\n", (->char)&e.kind - (->char)&e);
    printf("sizeof(kind) = %zu\n", sizeof(e.kind));
    printf("offset left = %td\n", (->char)&e.binary.left - (->char)&e);
    printf("sizeof(left) = %zu\n", sizeof(e.binary.left));
    printf("offset right = %td\n", (->char)&e.binary.right - (->char)&e);
    printf("sizeof(right) = %zu\n", sizeof(e.binary.right));
    printf("offset decimal = %td\n", (->char)&e.primary.decimal - (->char)&e);
    printf("sizeof(decimal) = %zu\n", sizeof(e.primary.decimal));
}
```

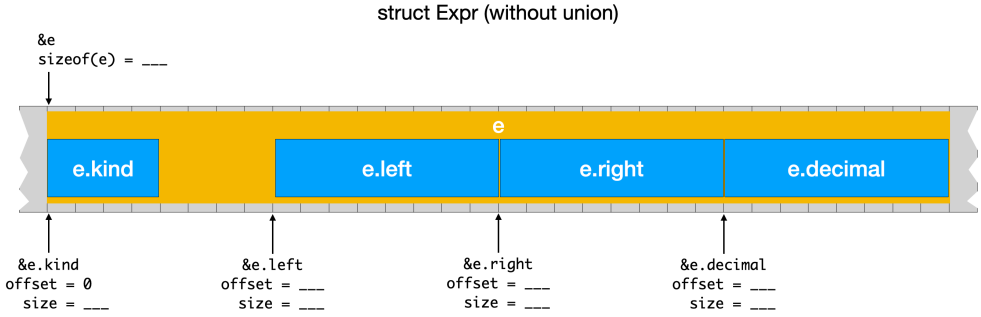
The **offset** of a member tells us how many bytes its address is away from the beginning of the complete `Expr` object. For example,

```
(->char)&e.binary.left - (->char)&e
```

computes the offset of `left`. We cast both addresses to `->char`, because a `char` occupies one byte and the resulting pointer difference therefore directly gives us the *distance in bytes*.

Tasks

1. Run `make` to compile the new test program and then execute `./xtest_expr_layout`. Use its output to complete the memory-layout sketch below. For each member, enter the **offset** and **size** you observe on your machine. Also enter `sizeof(Expr)` in the figure.



2. Look carefully at the layout. There may be some unused space between member `kind` and `binary.left`. Why might the compiler leave such a gap instead of placing left immediately after kind?

A Bit About Padding

The gap you observed is called padding. The compiler may insert unused bytes between members so that objects such as pointers start at suitably aligned memory addresses. The exact amount of padding depends on the types, the target architecture, and the compiler. For now, the important point is simply: the size of a struct is not necessarily the sum of the sizes of its members.

© Michael C. Lehn | CC BY-SA 4.0

Step 4 — Letting Members Share Memory

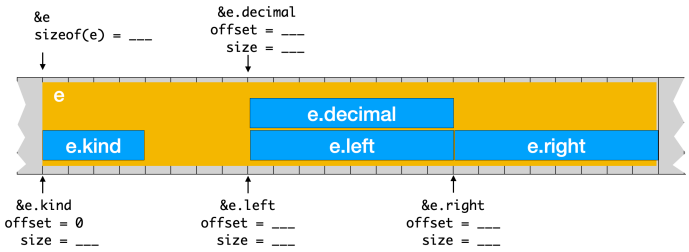
Our current `Expr` representation reserves separate memory for binary and primary, although an expression node never needs both representations at the same time.
Let us see what happens if we place them in a `union`. In `expr.hdr`, change the declaration of `Expr` to

```
struct Expr
{
    union {
        kind: ExprKind;
        binary:     struct Binary {
            left, right: ->Expr;
        };
        primary: struct Primary {
            decimal: ->UStr;
        };
    };
};
```

Tasks

1. Run make. Since `expr.hdr` has changed, make should rebuild everything that depends on this header — in particular our new `xtest_expr_layout` test program.

   Run `./xtest_expr_layout` again and use its output to complete the new memory-layout sketch below. Enter the **offset** and **size** of each member as well as `sizeof(Expr)`. If the sketch does not quite match what you observe — for example, if some padding is missing or a field is in the wrong place — correct it accordingly.



   Compare this layout with the one from Step 3. Which members now start at the same address, and how has `sizeof(Expr)` changed?
2. Extend `Expr` with the node types needed for the next parts of our compiler project:

```
struct Expr
{
    kind: ExprKind;
    union {
        unary: struct Unary {
            child: ->Expr;
        };
        binary: struct Binary {
            left, right: ->Expr;
        };
        primary: struct Primary {
            decimal: ->UStr;
            identifier: ->UStr;
            call: struct Call {
                fnName: ->UStr;
                argListLen: unsigned;
                argList: ->Expr;
            };
        };
        arg: struct ArgList {
            next: ->Expr;
            expr: ->Expr;
            pos: unsigned;
        };
    };
};
```

   Run `make` and then `./xtest_expr_layout` again. This time, draw the complete memory layout of `Expr` yourself, including all members, offsets, sizes, and padding. Also determine `sizeof(Expr)`.

© Michael C. Lehn | CC BY-SA 4.0

Step 5 — Function Calls in Expression Trees (Preparation)

So far, our expression trees can represent arithmetic expressions. Let us now extend them so that they can also represent function calls with a list of arguments.
As usual, we first write a small test program that describes what we want to achieve. Create `xtest_expr_fcall.abc`:

```
@ <stdio.hdr>
@ "expr.hdr"

fn main()
{
    local expr: -> Expr;

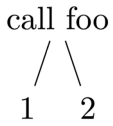
    expr = createCallExpr(UStrCreate("foo"));
    addArgExpr(expr, createIntegerExpr(UStrCreate("4")));
    addArgExpr(expr, createIntegerExpr(UStrCreate("2")));

    printExprTree(expr);
    printf("\n");
    releaseExpr(expr);
}
```

Our goal is to represent the function call `foo(4, 2)` so that `printExprTree` produces

```
[call foo [4][2]]
```

The corresponding expression tree should look like this:



Step 6 — Function Calls in Expression Trees

1. Try to compile everything with `make`. The first error should be

   ```
         expr = createCallExpr(UStrCreate("foo"));
                ^^^^^^^^^^^^^^^
   xtest_expr_fcall.abc:8.12-8.26: error: undefined identifier
   ```

   The compiler is telling us exactly what is missing: `createCallExpr` has not been declared anywhere.
2. Add the following declarations to `expr.hdr`. We already anticipate that `addArgExpr` will be needed as well:

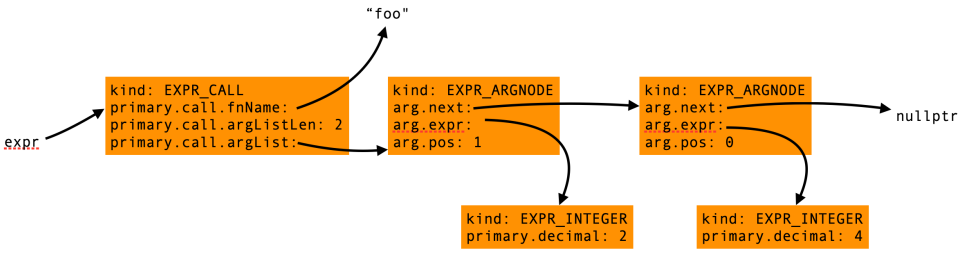
   ```
   extern fn createCallExpr(fnName: ->UStr): ->Expr;
   extern fn addArgExpr(callExpr: ->Expr, argExpr: ->Expr);
   ```

   Run `make` again. The linker error tells us what to do next: we need to implement the two functions.
   Before we can implement them, however, we first need to understand what the corresponding `Expr` data structure should actually look like. We will do that now and leave the implementation for Step 6.

   So grab pencil and paper!

What Comes Next

The sketch below shows what our expression tree will look like once we have completed the next step. Notice that we deliberately omit many details discussed earlier to focus on what matters here. The data structure has become more complex, but not more complicated — we just need to keep track of what points where.



© Michael C. Lehn | CC BY-SA 4.0

Step 7 — Adding Function Calls

We need two new kinds of nodes. Extend `ExprKind` in `expr.hdr` as follows:

```
enum ExprKind
{
    EXPR_BINARY,
    EXPR_ADD = EXPR_BINARY,
    EXPR_SUB,
    EXPR_MUL,
    EXPR_DIV,
    EXPR_MOD,
    EXPR_BINARY_END,

    EXPR_PRIMARY = EXPR_BINARY_END,
    EXPR_INTEGER,
    EXPR_CALL,
    EXPR_PRIMARY_END,

    EXPR_ARGNODE,
};
```

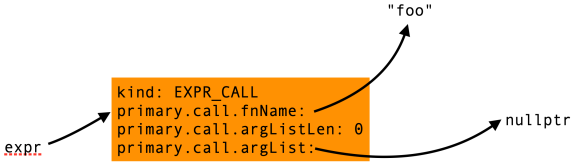
An `EXPR_CALL` node represents a function call, and the value returned by the function is the value of the expression. It therefore clearly belongs to our group of **primary expressions**.
An `EXPR_ARGNODE`, on the other hand, is a node in the linked list of arguments. We do not introduce a separate group for it, but it is deliberately **neither a primary nor a binary expression**.

Tasks

1. After

   ```
   expr = createCallExpr(UStrCreate("foo"));
   ```

   a new `EXPR_CALL` node should have been created. We can sketch its state as follows:



   That means:
   - `primary.call.fnName` points to the unique string `"foo"`.
   - primary.call.argListLen keeps track of the number of arguments and is initially 0.
   - primary.call.argList points to the first argument node. Since there are no arguments yet, it is initialized to nullptr.

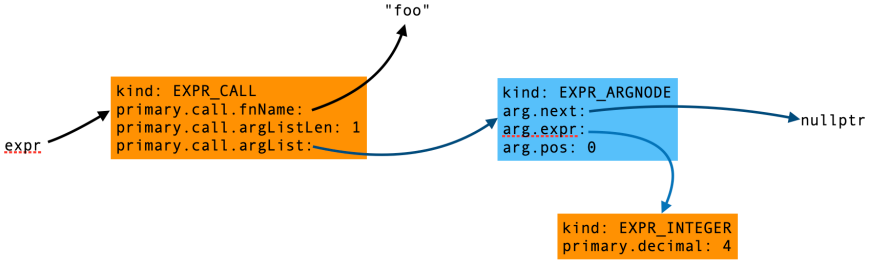
   Once we know exactly what we want, the implementation is straightforward:

   ```
   fn createCallExpr(fnName: ->UStr): ->Expr
   {
       local n: -> Expr = malloc(sizeof(*n));
       n->kind = EXPR_CALL;
       n->primary.call.fnName = fnName;
       n->primary.call.argListLen = 0;
       n->primary.call.argList = nullptr;
       return n;
   }
   ```

2. Next, consider what should happen when

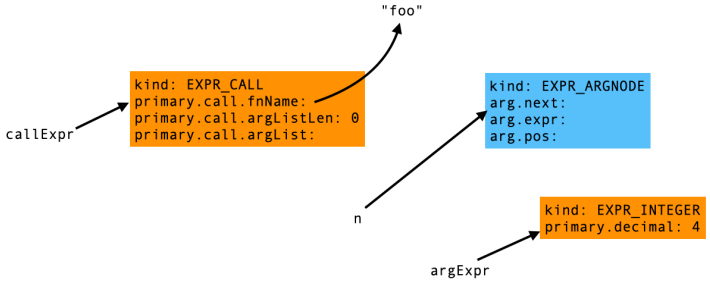
   ```
   addArgExpr(expr, createIntegerExpr(UStrCreate("4")))
   ```

   is executed. This is a bit tricky — give it a try! The solution is waiting for you on the next page.



© Michael C. Lehn | CC BY-SA 4.0

3. It helps to look at the problem from the right perspective — in this case, from inside function `addArgExpr`. The nested call `createIntegerExpr(UStrCreate("4"))` has already been evaluated, when the function is entered. Thus, `callExpr` points to the call node created before, while `argExpr` points to the new integer node:



From here, we only have to connect the pieces:
- Allocate a new node and set its kind to `EXPR_ARGNODE`.
- Let `arg.expr` point to the expression passed as `argExpr`.
- Store the current argument position in `arg.pos`, which is simply the current length of the argument list, `callExpr->primary.call.argListLen`.
- Insert the new node at the front of the argument list.
- Increment `callExpr->primary.call.argListLen`.

Expressed in code, this becomes:

```
fn addArgExpr(callExpr: ->Expr, argExpr: ->Expr)
{
    local n: -> Expr = malloc(sizeof(*n));
    n->kind = EXPR_ARGNODE;
    n->arg.expr = argExpr;
    n->arg.pos = callExpr->primary.call.argListLen;
    n->arg.next = callExpr->primary.call.argList;
    ++callExpr->primary.call.argListLen;
    callExpr->primary.call.argList = n;
}
```

Run `make` again. Everything should compile. However, running `./xtest_expr_fcall` gives us a runtime error:

```
expr.abc:61: Assertion '0' failed.
Abort trap: 6
```

4. The cause is quickly found: `printExprTree` does not yet know how to handle our new node kinds. Extend it as follows:

```
fn printExprTree(expr: ->Expr)
{
    if (expr->kind >= EXPR_PRIMARY && expr->kind < EXPR_PRIMARY_END) {
        switch (expr->kind) {
            // as before
            case EXPR_CALL:
                printf("[call %s ", expr->primary.call.fnName);
                printExprTree(expr->primary.call.argList);
                printf("]");
                break;
            default: assert(0 && "node kind not handled");
        }
    } else if (expr->kind >= EXPR_BINARY && expr->kind < EXPR_BINARY_END) {
        // as before
    } else if (expr->kind == EXPR_ARGNODE) {
        if (expr->arg.next) {
            printExprTree(expr->arg.next);
        }
        printExprTree(expr->arg.expr);
    } else {
        assert(0 && "node kind not handled");
    }
}
```

The code for `EXPR_ARGNODE` should look familiar: it is essentially the same pattern we used before to traverse a singly linked list in reverse order. Why do we need the reverse order here?

5. Our new nodes are not yet handled correctly by `releaseExpr`. Fix it so that the complete expression tree is released without memory leaks. ;-)

© Michael C. Lehn | CC BY-SA 4.0

Step 8 — Unary Expressions

To finish this worksheet, let us add unary expressions. Extend `ExprKind` in `expr.hdr` as follows:

```
enum ExprKind
{
    EXPR_UNARY,                         // new: begin
    EXPR_UNARY_MINUS = EXPR_UNARY,
    EXPR_UNARY_END,

    EXPR_BINARY = EXPR_UNARY_END,       // new: end
    EXPR_ADD = EXPR_BINARY,
    EXPR_SUB,
    EXPR_MUL,
    EXPR_DIV,
    EXPR_MOD,
    EXPR_BINARY_END,

    EXPR_PRIMARY = EXPR_BINARY_END,
    EXPR_INTEGER,
    EXPR_CALL,
    EXPR_PRIMARY_END,

    EXPR_ARGNODE,
};
```

`EXPR_UNARY_MINUS` will later represent negation in our compiler. It belongs to our new group of unary expressions, whose nodes have exactly one child.

We will need additional operators as our compiler grows, but we will not need additional groups of expression nodes. From now on, extending our expression trees follows a common pattern:

- Add a new `ExprKind` to the appropriate unary, binary, or primary group.
- Add the corresponding members to `Expr` if the new node needs additional data.
- Add a new constructor function, or adapt an existing one, *if necessary*.
- Extend `printExprTree` to handle the new node kind.
- `releaseExpr` already handles the three groups generically and therefore needs no further changes.

At this point, adding another expression kind is no longer really a design problem — it is mostly mechanical work. In fact, even an AI that has never seen our programming language before should be able to do it. ;-)

Tasks

Create a new test program `xtest_expr_unary.abc`:

```
@ <stdio.hdr>
@ "expr.hdr"

fn main()
{
    local expr: -> Expr;

    expr = createUnaryExpr(EXPR_UNARY_MINUS,
                           createIntegerExpr(UStrCreate("42")));
    printExprTree(expr);
    printf("\n");

    releaseExpr(expr);
}
```

To make sure that we all use the same interface, add the following declaration to `expr.hdr`:

```
extern fn createUnaryExpr(kind: ExprKind, child: ->Expr): ->Expr;
```

Now complete the implementation yourself:

1. Implement `createUnaryExpr`.
2. Extend `printExprTree` so that the test produces `[minus [42]]`.
3. Extend `releaseExpr` to correctly release unary expression trees.
4. Run `make` and verify the result with `./xtest_expr_unary`.

© Michael C. Lehn | CC BY-SA 4.0